

T.Y.B.Sc. (Computer Science) – Sem V

CS-305-MJ-P Operating System-I

Complete Practical Slip-wise Solution Booklet

Based on the uploaded SPPU 30-page practical examination slip PDF.

Important: The uploaded paper contains repeated questions. To keep the booklet practical for exam preparation, identical questions use the same reusable solution pattern, while each slip is listed separately.

Common formulas: $TAT = \text{Completion Time} - \text{Arrival Time}$; $WT = TAT - \text{CPU Burst}$. For Banker's algorithm, $\text{Need} = \text{Max} - \text{Allocation}$.

SLIP 1

Questions reproduced from page 1 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Banker's menu program

Use the Allocation/Max/Available values printed in the slip; Need is Max – Allocation.

```
#include <stdio.h>
int main() {
    int n,m,i,j,k,choice;
    int alloc[20][10], max[20][10], need[20][10], avail[10];

    printf("Enter processes and resources: ");
    scanf("%d%d",&n,&m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&max[i][j]);

    printf("Enter Available vector:\n");
    for(j=0;j<m;j++) scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(1){
        printf("\n1.Accept Available  2.Display Allocation/Max");
        printf("\n3.Display Need  4.Display Available  5.Safe Sequence  6.Exit\n");
        scanf("%d",&choice);
        if(choice==1){
            printf("Enter Available: ");
            for(j=0;j<m;j++) scanf("%d",&avail[j]);
        } else if(choice==2){
            printf("Allocation:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",alloc[i][j]); printf("\n"); }
            printf("Max:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",max[i][j]); printf("\n"); }
        } else if(choice==3){
            printf("Need:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",need[i][j]); printf("\n"); }
        } else if(choice==4){
            for(j=0;j<m;j++) printf("%d ",avail[j]);
            printf("\n");
        } else if(choice==5){
            int work[10],finish[20]={0},seq[20],cnt=0,found;
            for(j=0;j<m;j++) work[j]=avail[j];
            do{
                found=0;
                for(i=0;i<n;i++) if(!finish[i]){
                    int ok=1;
                    for(j=0;j<m;j++) if(need[i][j]>work[j]) ok=0;
                    if(ok){
                        for(j=0;j<m;j++) work[j]+=alloc[i][j];
                        finish[i]=1; seq[cnt++]=i; found=1;
                    }
                }
            }while(found);
            if(cnt==n){
                printf("Safe sequence: ");
                for(i=0;i<n;i++) printf("P%d%s",seq[i],i==n-1?"\n":" -> ");
            } else printf("System is NOT in safe state.\n");
        } else break;
    }
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 2

Questions reproduced from page 2 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Linked file allocation + FCFS

For the linked-allocation version, store the selected free blocks as the file's linked block list. FCFS disk scheduling uses the requests exactly in input order.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
typedef struct {char name[30]; int blocks[MAX],count;} File;
int main(){
    int n,i,ch,j,k,start; int used[MAX]={0}; File dir[MAX]; int nf=0;
    printf("Number of blocks: ");scanf("%d",&n);
    srand(1);
    for(i=0;i<n/5;i++) used[rand()%n]=1;
    while(1){
        printf("\n1.Show Bit Vector 2.Create New File 3.Show Directory 4.Delete File 5.Exit\n");
        scanf("%d",&ch);
        if(ch==1){for(i=0;i<n;i++)printf("%d ",used[i]);printf("\n");}
        else if(ch==2){
            char name[30];int m;printf("File name and blocks: ");scanf("%s%d",name,&m);
            printf("Enter %d free blocks (for linked/index simulation): ",m);
            if(nf>=MAX){printf("Directory full\n");continue;}
            strcpy(dir[nf].name,name);dir[nf].count=0;
            for(j=0;j<m;j++){scanf("%d",&k);if(k>=0&&k<n&&!used[k]){used[k]=1;dir[nf].blocks[dir[nf].count++]=k;}}
            nf++;
        } else if(ch==3){
            for(i=0;i<nf;i++){printf("%s: ",dir[i].name);for(j=0;j<dir[i].count;j++)printf("%d ",dir[i].blocks[j]);printf("\n");}
        } else if(ch==4){
            char name[30];scanf("%s",name);
            for(i=0;i<nf;i++)if(strcmp(dir[i].name,name)==0){for(j=0;j<dir[i].count;j++)used[dir[i].blocks[j]]=0;dir[i]=dir[--nf];break;}
        } else break;
    }
    return 0;
}
```

FCFS disk scheduling

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int n,req[100],head,i,j,total=0;
    printf("Enter number of requests: "); scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++) scanf("%d",&req[i]);
    printf("Enter initial head: "); scanf("%d",&head);

    printf("Service order: ");
    for(i=0;i<n;i++){
        printf("%d ",req[i]);
        total += abs(head-req[i]);
        head=req[i];
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 3

Questions reproduced from page 3 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Banker's algorithm

For the given A-D snapshot, enter the matrices exactly as printed; the program computes Need and the safe sequence.

```
#include <stdio.h>
int main() {
    int n,m,i,j,k,choice;
    int alloc[20][10], max[20][10], need[20][10], avail[10];

    printf("Enter processes and resources: ");
    scanf("%d%d",&n,&m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&max[i][j]);

    printf("Enter Available vector:\n");
    for(j=0;j<m;j++) scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(1){
        printf("\n1.Accept Available 2.Display Allocation/Max");
        printf("\n3.Display Need 4.Display Available 5.Safe Sequence 6.Exit\n");
        scanf("%d",&choice);
        if(choice==1){
            printf("Enter Available: ");
            for(j=0;j<m;j++) scanf("%d",&avail[j]);
        } else if(choice==2){
            printf("Allocation:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",alloc[i][j]); printf("\n"); }
            printf("Max:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",max[i][j]); printf("\n"); }
        } else if(choice==3){
            printf("Need:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",need[i][j]); printf("\n"); }
        } else if(choice==4){
            for(j=0;j<m;j++) printf("%d ",avail[j]);
            printf("\n");
        } else if(choice==5){
            int work[10],finish[20]={0},seq[20],cnt=0,found;
            for(j=0;j<m;j++) work[j]=avail[j];
            do{
                found=0;
                for(i=0;i<n;i++) if(!finish[i]){
                    int ok=1;
                    for(j=0;j<m;j++) if(need[i][j]>work[j]) ok=0;
                    if(ok){
                        for(j=0;j<m;j++) work[j]+=alloc[i][j];
                        finish[i]=1; seq[cnt++]=i; found=1;
                    }
                }
            }while(found);
            if(cnt==n){
                printf("Safe sequence: ");
                for(i=0;i<n;i++) printf("P%d%s",seq[i],i==n-1?"\n":" -> ");
            } else printf("System is NOT in safe state.\n");
        } else break;
    }
    return 0;
}
```

SSTF disk scheduling

```
#include <stdio.h>
#include <stdlib.h>
int main(){
```

```

int n, req[100], used[100]={0}, head, i, j, idx, dist, best, total=0;
scanf("%d", &n);
for(i=0; i<n; i++) scanf("%d", &req[i]);
scanf("%d", &head);
printf("Service order: ");
for(i=0; i<n; i++){
    idx=-1; best=1000000;
    for(j=0; j<n; j++) if(!used[j]){
        dist=abs(head-req[j]);
        if(dist<best){best=dist; idx=j;}
    }
    used[idx]=1; total+=abs(head-req[idx]); head=req[idx];
    printf("%d ", head);
}
printf("\nTotal head movement = %d\n", total);
return 0;
}

```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 4

Questions reproduced from page 4 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Banker's menu program

```
#include <stdio.h>
int main() {
    int n,m,i,j,k,choice;
    int alloc[20][10], max[20][10], need[20][10], avail[10];

    printf("Enter processes and resources: ");
    scanf("%d%d",&n,&m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&max[i][j]);

    printf("Enter Available vector:\n");
    for(j=0;j<m;j++) scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(1){
        printf("\n1.Accept Available  2.Display Allocation/Max");
        printf("\n3.Display Need  4.Display Available  5.Safe Sequence  6.Exit\n");
        scanf("%d",&choice);
        if(choice==1){
            printf("Enter Available: ");
            for(j=0;j<m;j++) scanf("%d",&avail[j]);
        } else if(choice==2){
            printf("Allocation:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",alloc[i][j]); printf("\n"); }
            printf("Max:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",max[i][j]); printf("\n"); }
        } else if(choice==3){
            printf("Need:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",need[i][j]); printf("\n"); }
        } else if(choice==4){
            for(j=0;j<m;j++) printf("%d ",avail[j]);
            printf("\n");
        } else if(choice==5){
            int work[10],finish[20]={0},seq[20],cnt=0,found;
            for(j=0;j<m;j++) work[j]=avail[j];
            do{
                found=0;
                for(i=0;i<n;i++) if(!finish[i]){
                    int ok=1;
                    for(j=0;j<m;j++) if(need[i][j]>work[j]) ok=0;
                    if(ok){
                        for(j=0;j<m;j++) work[j]+=alloc[i][j];
                        finish[i]=1; seq[cnt++]=i; found=1;
                    }
                }
            }while(found);
            if(cnt==n){
                printf("Safe sequence: ");
                for(i=0;i<n;i++) printf("P%d%s",seq[i],i==n-1?"\n":" -> ");
            } else printf("System is NOT in safe state.\n");
        } else break;
    }
    return 0;
}
```

SCAN disk scheduling

For this slip use dir=0 for Left.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=i+1;j<n;j++)if(a[i]>a[j]){t=a[i];a[i]=a[j];a[j]=t;}}
```

```

int main(){
    int n,a[100],head,dir,i,p=0,total=0;
    scanf("%d",&n); for(i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d%d",&head,&dir); /* dir: 0=left, 1=right */
    sort(a,n);
    printf("Service order: ");
    if(dir==0){
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }else{
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}

```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 5

Questions reproduced from page 5 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

FIFO page replacement

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],i,j,pos=0,fault=0,hit;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++) scanf("%d",&ref[i]);
    printf("Enter number of frames: "); scanf("%d",&f);
    for(i=0;i<f;i++) frame[i]=-1;

    for(i=0;i<n;i++){
        hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]) hit=1;
        if(!hit){ frame[pos]=ref[i]; pos=(pos+1)%f; fault++; }
        printf("%d: ",ref[i]);
        for(j=0;j<f;j++) printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault);
    return 0;
}
```

Resource-request / Banker check

The same Need calculation and safety test are the core of the resource-request algorithm.

```
#include <stdio.h>
int main() {
    int n,m,i,j,k,choice;
    int alloc[20][10], max[20][10], need[20][10], avail[10];

    printf("Enter processes and resources: ");
    scanf("%d%d",&n,&m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&max[i][j]);

    printf("Enter Available vector:\n");
    for(j=0;j<m;j++) scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(1){
        printf("\n1.Accept Available 2.Display Allocation/Max");
        printf("\n3.Display Need 4.Display Available 5.Safe Sequence 6.Exit\n");
        scanf("%d",&choice);
        if(choice==1){
            printf("Enter Available: ");
            for(j=0;j<m;j++) scanf("%d",&avail[j]);
        } else if(choice==2){
            printf("Allocation:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",alloc[i][j]); printf("\n"); }
            printf("Max:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",max[i][j]); printf("\n"); }
        } else if(choice==3){
            printf("Need:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",need[i][j]); printf("\n"); }
        } else if(choice==4){
            for(j=0;j<m;j++) printf("%d ",avail[j]);
            printf("\n");
        } else if(choice==5){
            int work[10],finish[20]={0},seq[20],cnt=0,found;
            for(j=0;j<m;j++) work[j]=avail[j];
            do{
                found=0;

```

```

    for(i=0;i<n;i++) if(!finish[i]){
        int ok=1;
        for(j=0;j<m;j++) if(need[i][j]>work[j]) ok=0;
        if(ok){
            for(j=0;j<m;j++) work[j]+=alloc[i][j];
            finish[i]=1; seq[cnt++]=i; found=1;
        }
    }
}while(found);
if(cnt==n){
    printf("Safe sequence: ");
    for(i=0;i<n;i++) printf("P%d%s",seq[i],i==n-1?"\n":" -> ");
} else printf("System is NOT in safe state.\n");
} else break;
}
return 0;
}

```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 6

Questions reproduced from page 6 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Sequential/contiguous file allocation

For contiguous allocation, create a file only when enough consecutive free blocks are available; adapt the Create option to check consecutiveness.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
typedef struct {char name[30]; int blocks[MAX],count;} File;
int main(){
    int n,i,ch,j,k,start; int used[MAX]={0}; File dir[MAX]; int nf=0;
    printf("Number of blocks: ");scanf("%d",&n);
    srand(1);
    for(i=0;i<n/5;i++) used[rand()%n]=1;
    while(1){
        printf("\n1.Show Bit Vector 2.Create New File 3.Show Directory 4.Delete File 5.Exit\n");
        scanf("%d",&ch);
        if(ch==1){for(i=0;i<n;i++)printf("%d ",used[i]);printf("\n");}
        else if(ch==2){
            char name[30];int m;printf("File name and blocks: ");scanf("%s%d",name,&m);
            printf("Enter %d free blocks (for linked/index simulation): ",m);
            if(nf>=MAX){printf("Directory full\n");continue;}
            strcpy(dir[nf].name,name);dir[nf].count=0;
            for(j=0;j<m;j++){scanf("%d",&k);if(k>=0&&k<n&&!used[k]){used[k]=1;dir[nf].blocks[dir[nf].count++]=k;}}
            nf++;
        } else if(ch==3){
            for(i=0;i<nf;i++){printf("%s: ",dir[i].name);for(j=0;j<dir[i].count;j++)printf("%d ",dir[i].blocks[j]);printf("\n");}
        } else if(ch==4){
            char name[30];scanf("%s",name);
            for(i=0;i<nf;i++){if(strcmp(dir[i].name,name)==0){for(j=0;j<dir[i].count;j++)used[dir[i].blocks[j]]=0;dir[i]=dir[--nf];break;}}
        } else break;
    }
    return 0;
}
```

C-SCAN disk scheduling

Disk size is 200 and direction is Right.

```
/* C-SCAN: dir=1 means right, disk size 0..diskSize-1 */
#include <stdio.h>
#include <stdlib.h>
void sort(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=i+1;j<n;j++){if(a[i]>a[j]){t=a[i];a[i]=a[j];a[j]=t;}}
int main(){
    int n,a[100],head,diskSize=200,i,total=0,dir;
    scanf("%d",&n);for(i=0;i<n;i++)scanf("%d",&a[i]);
    scanf("%d",&head); scanf("%d",&dir); sort(a,n);
    printf("Service order: ");
    if(dir==1){
        for(i=0;i<n;i++) if(a[i]>=head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        total += (diskSize-1-head); head=diskSize-1;
        total += diskSize-1; head=0;
        for(i=0;i<n;i++) if(a[i]<head+1){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    } else {
        for(i=n-1;i>=0;i--) if(a[i]<=head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        total += head; head=0;
        total += diskSize-1; head=diskSize-1;
        for(i=n-1;i>=0;i--) if(a[i]>head-1){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 7

Questions reproduced from page 7 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Resource-request / Banker safety

```
#include <stdio.h>
int main() {
    int n,m,i,j,k,choice;
    int alloc[20][10], max[20][10], need[20][10], avail[10];

    printf("Enter processes and resources: ");
    scanf("%d%d",&n,&m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&max[i][j]);

    printf("Enter Available vector:\n");
    for(j=0;j<m;j++) scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(1){
        printf("\n1.Accept Available  2.Display Allocation/Max");
        printf("\n3.Display Need  4.Display Available  5.Safe Sequence  6.Exit\n");
        scanf("%d",&choice);
        if(choice==1){
            printf("Enter Available: ");
            for(j=0;j<m;j++) scanf("%d",&avail[j]);
        } else if(choice==2){
            printf("Allocation:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",alloc[i][j]); printf("\n"); }
            printf("Max:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",max[i][j]); printf("\n"); }
        } else if(choice==3){
            printf("Need:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",need[i][j]); printf("\n"); }
        } else if(choice==4){
            for(j=0;j<m;j++) printf("%d ",avail[j]);
            printf("\n");
        } else if(choice==5){
            int work[10],finish[20]={0},seq[20],cnt=0,found;
            for(j=0;j<m;j++) work[j]=avail[j];
            do{
                found=0;
                for(i=0;i<n;i++) if(!finish[i]){
                    int ok=1;
                    for(j=0;j<m;j++) if(need[i][j]>work[j]) ok=0;
                    if(ok){
                        for(j=0;j<m;j++) work[j]+=alloc[i][j];
                        finish[i]=1; seq[cnt++]=i; found=1;
                    }
                }
            } while(found);
            if(cnt==n){
                printf("Safe sequence: ");
                for(i=0;i<n;i++) printf("P%d%s",seq[i],i==n-1?"\n":" -> ");
            } else printf("System is NOT in safe state.\n");
        } else break;
    }
    return 0;
}
```

nice() system call

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
```

```

int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child PID=%d, priority before nice=%d\n",getpid(),nice(0));
        errno=0;
        int r=nice(-5); /* lower nice value = higher priority; may require privilege */
        if(r==-1 && errno!=0) perror("nice");
        printf("Child nice value after request=%d\n",nice(0));
    }else{
        wait(NULL);
        printf("Parent finished.\n");
    }
    return 0;
}

```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 8

Questions reproduced from page 8 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Index file allocation

For an index allocation implementation, reserve one index block and store the file's data-block numbers in the index list.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
typedef struct {char name[30]; int blocks[MAX],count;} File;
int main(){
    int n,i,ch,j,k,start; int used[MAX]={0}; File dir[MAX]; int nf=0;
    printf("Number of blocks: ");scanf("%d",&n);
    srand(1);
    for(i=0;i<n/5;i++) used[rand()%n]=1;
    while(1){
        printf("\n1.Show Bit Vector 2.Create New File 3.Show Directory 4.Delete File 5.Exit\n");
        scanf("%d",&ch);
        if(ch==1){for(i=0;i<n;i++)printf("%d ",used[i]);printf("\n");}
        else if(ch==2){
            char name[30];int m;printf("File name and blocks: ");scanf("%s%d",name,&m);
            printf("Enter %d free blocks (for linked/index simulation): ",m);
            if(nf==MAX){printf("Directory full\n");continue;}
            strcpy(dir[nf].name,name);dir[nf].count=0;
            for(j=0;j<m;j++){scanf("%d",&k);if(k>=0&&k<n&&!used[k]){used[k]=1;dir[nf].blocks[dir[nf].count++]=k;}}
            nf++;
        } else if(ch==3){
            for(i=0;i<nf;i++){printf("%s: ",dir[i].name);for(j=0;j<dir[i].count;j++)printf("%d ",dir[i].blocks[j]);printf("\n");}
        } else if(ch==4){
            char name[30];scanf("%s",name);
            for(i=0;i<nf;i++)if(strcmp(dir[i].name,name)==0){for(j=0;j<dir[i].count;j++)used[dir[i].blocks[j]]=0;dir[i]=dir[--nf];break;}
        } else break;
    }
    return 0;
}
```

nice() system call

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child PID=%d, priority before nice=%d\n",getpid(),nice(0));
        errno=0;
        int r=nice(-5); /* lower nice value = higher priority; may require privilege */
        if(r== -1 && errno!=0) perror("nice");
        printf("Child nice value after request=%d\n",nice(0));
    }else{
        wait(NULL);
        printf("Parent finished.\n");
    }
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 9

Questions reproduced from page 9 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Optimal page replacement

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],i,j,k,hit,fault=0,idx,far,next;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++) scanf("%d",&ref[i]);
    printf("Enter number of frames: "); scanf("%d",&f);
    for(i=0;i<f;i++) frame[i]=-1;

    for(i=0;i<n;i++){
        hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]) hit=1;
        if(!hit){
            fault++;
            idx=-1;
            for(j=0;j<f;j++) if(frame[j]==-1){idx=j;break;}
            if(idx==-1){
                far=-1;
                for(j=0;j<f;j++){
                    next=9999;
                    for(k=i+1;k<n;k++) if(ref[k]==frame[j]){next=k;break;}
                    if(next>far){far=next;idx=j;}
                }
            }
            frame[idx]=ref[i];
        }
        printf("%d: ",ref[i]);
        for(j=0;j<f;j++) printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault);
    return 0;
}
```

Non-preemptive SJF

```
#include <stdio.h>
int main(){
    int n,i,j,t=0,done=0,idx,temp; scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],used[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);used[i]=0;}
    printf("Gantt: ");
    while(done<n){
        idx=-1;
        for(i=0;i<n;i++) if(!used[i]&&at[i]<=t&&(idx==-1||bt[i]<bt[idx])) idx=i;
        if(idx==-1){t++;continue;}
        printf("P%d ",idx); t+=bt[idx]; ct[idx]=t; tat[idx]=ct[idx]-at[idx]; wt[idx]=tat[idx]-bt[idx]; used[idx]=1;done++;
    }
    float aw=0,aa=0;printf("\nPID AT BT CT TAT WT\n");
    for(i=0;i<n;i++){printf("P%d %d %d %d %d %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 10

Questions reproduced from page 10 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Round Robin

```
#include <stdio.h>
int main(){
    int n,q,i,t=0,done=0;scanf("%d%d",&n,&q);
    int at[n],bt[n],rem[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int progress=0;
        for(i=0;i<n;i++) if(rem[i]>0 && at[i]<=t){
            progress=1;printf(" P%d ",i);
            int x=rem[i]<q?rem[i]:q; t+=x;rem[i]-=x;
            if(rem[i]==0){ct[i]=t;tat[i]=ct[i]-at[i];wt[i]=tat[i]-bt[i];done++;}
        }
        if(!progress)t++;
    }
    printf("\n");float aw=0,aa=0;for(i=0;i<n;i++){printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

LOOK disk scheduling

For this slip use dir=0 for Left.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=i+1;j<n;j++)if(a[i]>a[j]){t=a[i];a[i]=a[j];a[j]=t;}}
int main(){
    int n,a[100],head,dir,i,total=0;
    scanf("%d",&n); for(i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d%d",&head,&dir); sort(a,n);
    printf("Service order: ");
    if(dir==0){
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }else{
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 11

Questions reproduced from page 11 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Banker's menu program

```
#include <stdio.h>
int main() {
    int n,m,i,j,k,choice;
    int alloc[20][10], max[20][10], need[20][10], avail[10];

    printf("Enter processes and resources: ");
    scanf("%d%d",&n,&m);

    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&alloc[i][j]);

    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++) for(j=0;j<m;j++) scanf("%d",&max[i][j]);

    printf("Enter Available vector:\n");
    for(j=0;j<m;j++) scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    while(1){
        printf("\n1.Accept Available  2.Display Allocation/Max");
        printf("\n3.Display Need  4.Display Available  5.Safe Sequence  6.Exit\n");
        scanf("%d",&choice);
        if(choice==1){
            printf("Enter Available: ");
            for(j=0;j<m;j++) scanf("%d",&avail[j]);
        } else if(choice==2){
            printf("Allocation:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",alloc[i][j]); printf("\n"); }
            printf("Max:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",max[i][j]); printf("\n"); }
        } else if(choice==3){
            printf("Need:\n");
            for(i=0;i<n;i++){ for(j=0;j<m;j++) printf("%d ",need[i][j]); printf("\n"); }
        } else if(choice==4){
            for(j=0;j<m;j++) printf("%d ",avail[j]);
            printf("\n");
        } else if(choice==5){
            int work[10],finish[20]={0},seq[20],cnt=0,found;
            for(j=0;j<m;j++) work[j]=avail[j];
            do{
                found=0;
                for(i=0;i<n;i++) if(!finish[i]){
                    int ok=1;
                    for(j=0;j<m;j++) if(need[i][j]>work[j]) ok=0;
                    if(ok){
                        for(j=0;j<m;j++) work[j]+=alloc[i][j];
                        finish[i]=1; seq[cnt++]=i; found=1;
                    }
                }
            } while(found);
            if(cnt==n){
                printf("Safe sequence: ");
                for(i=0;i<n;i++) printf("P%d%s",seq[i],i==n-1?"\n":" -> ");
            } else printf("System is NOT in safe state.\n");
        } else break;
    }
    return 0;
}
```

Optimal page replacement

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],i,j,k,hit,fault=0,idx,far,next;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
```

```

for(i=0;i<n;i++) scanf("%d",&ref[i]);
printf("Enter number of frames: "); scanf("%d",&f);
for(i=0;i<f;i++) frame[i]=-1;

for(i=0;i<n;i++){
    hit=0;
    for(j=0;j<f;j++) if(frame[j]==ref[i]) hit=1;
    if(!hit){
        fault++;
        idx=-1;
        for(j=0;j<f;j++) if(frame[j]==-1){idx=j;break;}
        if(idx==-1){
            far=-1;
            for(j=0;j<f;j++){
                next=9999;
                for(k=i+1;k<n;k++) if(ref[k]==frame[j]){next=k;break;}
                if(next>far){far=next;idx=j;}
            }
        }
        frame[idx]=ref[i];
    }
    printf("%d: ",ref[i]);
    for(j=0;j<f;j++) printf("%d ",frame[j]);
    printf(hit?" Hit\n":" Fault\n");
}
printf("Total page faults = %d\n",fault);
return 0;
}

```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 12

Questions reproduced from page 12 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

FCFS CPU scheduling

```
#include <stdio.h>
int main(){
    int n,i,j,t=0; printf("Enter n: ");scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){printf("AT and CPU burst of P%d: ",i);scanf("%d%d",&at[i],&bt[i]);}
    for(i=0;i<n;i++){ if(t<at[i]) t=at[i]; t+=bt[i]; ct[i]=t; tat[i]=ct[i]-at[i]; wt[i]=tat[i]-bt[i];}
    printf("Gantt: |");for(i=0;i<n;i++)printf(" P%d |",i);
    printf("\nPID AT BT CT TAT WT\n");
    float aw=0,atot=0;
    for(i=0;i<n;i++){printf("P%d %d %d %d %d %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);aw+=wt[i];atot+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,atot/n); return 0;
}
```

fork + bubble/insertion sorting

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
void bubble(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=0;j<n-i-1;j++){if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}}
void insertion(int a[],int n){int i,j,key;for(i=1;i<n;i++){key=a[i];j=i-1;while(j>=0&& a[j]>key){a[j+1]=a[j];j--;}a[j+1]=key;}}
void print(int a[],int n){int i;for(i=0;i<n;i++)printf("%d ",a[i]);printf("\n");}
int main(){
    int n,a[100],b[100],i; scanf("%d",&n);
    for(i=0;i<n;i++){scanf("%d",&a[i]);b[i]=a[i];}
    pid_t p=fork();
    if(p==0){insertion(b,n);printf("Child - Insertion sort: ");print(b,n);}
    else{bubble(a,n);printf("Parent - Bubble sort: ");print(a,n);wait(NULL);}
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 13

Questions reproduced from page 13 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Non-preemptive SJF

```
#include <stdio.h>
int main(){
    int n,i,j,t=0,done=0,idx,temp; scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],used[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);used[i]=0;}
    printf("Gantt: ");
    while(done<n){
        idx=-1;
        for(i=0;i<n;i++) if(!used[i]&&at[i]<=t&&(idx== -1||bt[i]<bt[idx])) idx=i;
        if(idx== -1){t++;continue;}
        printf("P%d ",idx); t+=bt[idx]; ct[idx]=t; tat[idx]=ct[idx]-at[idx]; wt[idx]=tat[idx]-bt[idx]; used[idx]=1;done++;
    }
    float aw=0,aa=0;printf("\nPID AT BT CT TAT WT\n");
    for(i=0;i<n;i++){printf("P%d %d %d %d %d %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

LOOK disk scheduling

For this slip use dir=0 for Left.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=i+1;j<n;j++)if(a[i]>a[j]){t=a[i];a[i]=a[j];a[j]=t;}}
int main(){
    int n,a[100],head,dir,i,total=0;
    scanf("%d",&n); for(i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d%d",&head,&dir); sort(a,n);
    printf("Service order: ");
    if(dir==0){
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }else{
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 14

Questions reproduced from page 14 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

fork + execve + binary search

```
/* Main program: sort, then exec child program with sorted values */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void bubble(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=0;j<n-i-1;j++)if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}}
int main(int argc,char *argv[]){
    int n,i; scanf("%d",&n); int a[n]; char *args[105];
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    bubble(a,n);
    char key[20]; scanf("%s",key);
    args[0]="./search"; for(i=0;i<n;i++){args[i+1]=malloc(20);sprintf(args[i+1],"%d",a[i]);}
    args[n+1]=key; args[n+2]=NULL;
    pid_t p=fork();
    if(p==0) execve(args[0],args,NULL);
    else wait(NULL);
    return 0;
}

/* search.c
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char *argv[]){
    int n=argc-2,key=atoi(argv[argc-1]),l=0,r=n-1,mid;
    while(l<=r){mid=(l+r)/2;int x=atoi(argv[mid+1]);
        if(x==key){printf("Found at position %d\n",mid+1);return 0;}
        if(x<key)l=mid+1;else r=mid-1;
    }
    printf("Not found\n"); return 0;
} */
```

FIFO page replacement

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],i,j,pos=0,fault=0,hit;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++) scanf("%d",&ref[i]);
    printf("Enter number of frames: "); scanf("%d",&f);
    for(i=0;i<f;i++) frame[i]=-1;

    for(i=0;i<n;i++){
        hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]) hit=1;
        if(!hit){ frame[pos]=ref[i]; pos=(pos+1)%f; fault++; }
        printf("%d: ",ref[i]);
        for(j=0;j<f;j++) printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 15

Questions reproduced from page 15 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Orphan process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child: PID=%d, Parent PID=%d\n",getpid(),getppid());
        sleep(5);
        printf("Child after parent termination: PID=%d, New Parent PID=%d\n",getpid(),getppid());
    }else{
        printf("Parent: PID=%d, Child PID=%d\n",getpid(),pid);
        sleep(1);
        printf("Parent terminates.\n");
        exit(0);
    }
    return 0;
}
```

Preemptive Priority

```
#include <stdio.h>
int main(){
    int n,i,t=0,done=0;scanf("%d",&n);
    int at[n],bt[n],rem[n],pr[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d%d",&at[i],&bt[i],&pr[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int idx=-1;
        for(i=0;i<n;i++) if(at[i]<=t&&rem[i]>0&&(idx==-1||pr[i]<pr[idx])) idx=i;
        if(idx==-1){t++;continue;}
        printf("|P%d ",idx);rem[idx]--;t++;
        if(rem[idx]==0){ct[idx]=t;tat[idx]=ct[idx]-at[idx];wt[idx]=tat[idx]-bt[idx];done++;}
    }
    printf("\n");for(i=0;i<n;i++)printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 16

Questions reproduced from page 16 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

LRU page replacement – counter concept

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],last[20],i,j,t=0,fault=0,hit,idx,min;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++) scanf("%d",&ref[i]);
    printf("Enter number of frames: "); scanf("%d",&f);
    for(i=0;i<f;i++) frame[i]=-1,last[i]=-1;

    for(i=0;i<n;i++){
        t++; hit=0; idx=-1;
        for(j=0;j<f;j++) if(frame[j]==ref[i]){
            hit=1; last[j]=t;
        }
        if(!hit){
            fault++;
            for(j=0;j<f;j++) if(frame[j]==-1){ idx=j; break; }
            if(idx==-1){
                min=last[0]; idx=0;
                for(j=1;j<f;j++) if(last[j]<min){min=last[j];idx=j;}
            }
            frame[idx]=ref[i]; last[idx]=t;
        }
        printf("%d: ",ref[i]);
        for(j=0;j<f;j++) printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault);
    return 0;
}
```

Shell + count command

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(){
    char line[200],*args[20];
    while(1){
        int n=0,i;
        printf("$ "); fflush(stdout);
        if(!fgets(line,sizeof(line),stdin)) break;
        line[strcspn(line,"\n")]=0;
        if(strcmp(line,"exit")==0) break;

        char *p=strtok(line," ");
        while(p && n<19){args[n++]=p;p=strtok(NULL," ");}
        args[n]=NULL;
        if(n==0) continue;

        if(strcmp(args[0],"count")==0 && n==3){
            FILE *fp=fopen(args[2],"r"); int ch,c=0,w=0,l=0,inword=0;
            if(!fp){perror("fopen");continue;}
            while((ch=fgetc(fp))!=EOF){
                c++; if(ch=='\n')l++;
                if(ch==' '||ch=='\n'||ch=='\t') inword=0;
                else if(!inword){w++;inword=1;}
            }
            fclose(fp);
            if(strcmp(args[1],"c")==0) printf("Characters = %d\n",c);
            else if(strcmp(args[1],"w")==0) printf("Words = %d\n",w);
            else if(strcmp(args[1],"l")==0) printf("Lines = %d\n",l);
            else printf("Invalid count option\n");
            continue;
        }

        pid_t pid=fork();
```

```
    if(pid==0){execvp(args[0],args);perror("exec");exit(1);}
    else wait(NULL);
}
return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 17

Questions reproduced from page 17 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Non-preemptive SJF

```
#include <stdio.h>
int main(){
    int n,i,j,t=0,done=0,idx,temp; scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],used[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);used[i]=0;}
    printf("Gantt: ");
    while(done<n){
        idx=-1;
        for(i=0;i<n;i++) if(!used[i]&&at[i]<=t&&(idx== -1||bt[i]<bt[idx])) idx=i;
        if(idx== -1){t++;continue;}
        printf("P%d ",idx); t+=bt[idx]; ct[idx]=t; tat[idx]=ct[idx]-at[idx]; wt[idx]=tat[idx]-bt[idx]; used[idx]=1;done++;
    }
    float aw=0,aa=0;printf("\nPID AT BT CT TAT WT\n");
    for(i=0;i<n;i++){printf("P%d %d %d %d %d %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

Sequential/contiguous file allocation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
typedef struct {char name[30]; int blocks[MAX],count;} File;
int main(){
    int n,i,ch,j,k,start; int used[MAX]={0}; File dir[MAX]; int nf=0;
    printf("Number of blocks: ");scanf("%d",&n);
    srand(1);
    for(i=0;i<n/5;i++) used[rand()%n]=1;
    while(1){
        printf("\n1.Show Bit Vector 2.Create New File 3.Show Directory 4.Delete File 5.Exit\n");
        scanf("%d",&ch);
        if(ch==1){for(i=0;i<n;i++)printf("%d ",used[i]);printf("\n");}
        else if(ch==2){
            char name[30];int m;printf("File name and blocks: ");scanf("%s%d",name,&m);
            printf("Enter %d free blocks (for linked/index simulation): ",m);
            if(nf>=MAX){printf("Directory full\n");continue;}
            strcpy(dir[nf].name,name);dir[nf].count=0;
            for(j=0;j<m;j++){scanf("%d",&k);if(k>=0&&k<n&&!used[k]){used[k]=1;dir[nf].blocks[dir[nf].count++]=k;}}
            nf++;
        } else if(ch==3){
            for(i=0;i<nf;i++){printf("%s: ",dir[i].name);for(j=0;j<dir[i].count;j++)printf("%d ",dir[i].blocks[j]);printf("\n");}
        } else if(ch==4){
            char name[30];scanf("%s",name);
            for(i=0;i<nf;i++)if(strcmp(dir[i].name,name)==0){for(j=0;j<dir[i].count;j++)used[dir[i].blocks[j]]=0;dir[i]=dir[--nf];break;}
        } else break;
    }
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 18

Questions reproduced from page 18 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Preemptive SJF

```
#include <stdio.h>
int main(){
    int n,i,t=0,done=0;scanf("%d",&n);int at[n],bt[n],rem[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int idx=-1;
        for(i=0;i<n;i++){if(at[i]<=t&&rem[i]>0&&(idx== -1||rem[i]<rem[idx]))idx=i;
            if(idx== -1){t++;continue;}
            printf("|P%d ",idx);rem[idx]--;t++;
            if(rem[idx]==0){ct[idx]=t;tat[idx]=ct[idx]-at[idx];wt[idx]=tat[idx]-bt[idx];done++;}
        }
        printf("\n");for(i=0;i<n;i++){printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);return 0;
    }
}
```

Orphan process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child: PID=%d, Parent PID=%d\n",getpid(),getppid());
        sleep(5);
        printf("Child after parent termination: PID=%d, New Parent PID=%d\n",getpid(),getppid());
    }else{
        printf("Parent: PID=%d, Child PID=%d\n",getpid(),pid);
        sleep(1);
        printf("Parent terminates.\n");
        exit(0);
    }
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 19

Questions reproduced from page 19 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Preemptive Priority

```
#include <stdio.h>
int main(){
    int n,i,t=0,done=0;scanf("%d",&n);
    int at[n],bt[n],rem[n],pr[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d%d%d",&at[i],&bt[i],&pr[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int idx=-1;
        for(i=0;i<n;i++) if(at[i]<=t&&rem[i]>0&&(idx== -1||pr[i]<pr[idx])) idx=i;
        if(idx== -1){t++;continue;}
        printf("P%d ",idx);rem[idx]--;t++;
        if(rem[idx]==0){ct[idx]=t;tat[idx]=ct[idx]-at[idx];wt[idx]=tat[idx]-bt[idx];done++;}
    }
    printf("\n");for(i=0;i<n;i++)printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);return 0;
}
```

nice() system call

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child PID=%d, priority before nice=%d\n",getpid(),nice(0));
        errno=0;
        int r=nice(-5); /* lower nice value = higher priority; may require privilege */
        if(r== -1 && errno!=0) perror("nice");
        printf("Child nice value after request=%d\n",nice(0));
    }else{
        wait(NULL);
        printf("Parent finished.\n");
    }
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 20

Questions reproduced from page 20 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

LOOK disk scheduling

For this slip use dir=1 for Right.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=i+1;j<n;j++)if(a[i]>a[j]){t=a[i];a[i]=a[j];a[j]=t;}}
int main(){
    int n,a[100],head,dir,i,total=0;
    scanf("%d",&n); for(i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d%d",&head,&dir); sort(a,n);
    printf("Service order: ");
    if(dir==0){
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }else{
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}
```

LRU page replacement

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],last[20],i,j,t=0,fault=0,hit,idx,min;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++) scanf("%d",&ref[i]);
    printf("Enter number of frames: "); scanf("%d",&f);
    for(i=0;i<f;i++) frame[i]=-1,last[i]=-1;

    for(i=0;i<n;i++){
        t++; hit=0; idx=-1;
        for(j=0;j<f;j++) if(frame[j]==ref[i]){
            hit=1; last[j]=t;
        }
        if(!hit){
            fault++;
            for(j=0;j<f;j++) if(frame[j]==-1){ idx=j; break; }
            if(idx==-1){
                min=last[0]; idx=0;
                for(j=1;j<f;j++) if(last[j]<min){min=last[j];idx=j;}
            }
            frame[idx]=ref[i]; last[idx]=t;
        }
        printf("%d: ",ref[i]);
        for(j=0;j<f;j++) printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 21

Questions reproduced from page 21 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

MFU page replacement

For MFU, replace the page with the highest reference frequency. The generic LRU program is not an MFU implementation; use the MFU logic below.

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],freq[20],age[20],i,j,hit,idx,minAge,fault=0,t=0;
    scanf("%d",&n); for(i=0;i<n;i++)scanf("%d",&ref[i]); scanf("%d",&f);
    for(i=0;i<f;i++){frame[i]=-1;freq[i]=0;age[i]=0;}
    for(i=0;i<n;i++){
        t++; hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]){hit=1;freq[j]++;age[j]=t;break;}
        if(!hit){
            fault++;idx=-1;
            for(j=0;j<f;j++)if(frame[j]==-1){idx=j;break;}
            if(idx==-1){
                idx=0;
                for(j=1;j<f;j++)
                    if(freq[j]>freq[idx] || (freq[j]==freq[idx]&&age[j]<age[idx])) idx=j;
            }
            frame[idx]=ref[i];freq[idx]=1;age[idx]=t;
        }
        printf("%d: ",ref[i]);for(j=0;j<f;j++)printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault); return 0;
}
```

nice() system call

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child PID=%d, priority before nice=%d\n",getpid(),nice(0));
        errno=0;
        int r=nice(-5); /* lower nice value = higher priority; may require privilege */
        if(r==-1 && errno!=0) perror("nice");
        printf("Child nice value after request=%d\n",nice(0));
    }else{
        wait(NULL);
        printf("Parent finished.\n");
    }
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 22

Questions reproduced from page 22 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Linked file allocation

Use the file block list as linked blocks; Delete releases all blocks.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100
typedef struct {char name[30]; int blocks[MAX],count;} File;
int main(){
    int n,i,ch,j,k,start; int used[MAX]={0}; File dir[MAX]; int nf=0;
    printf("Number of blocks: ");scanf("%d",&n);
    srand(1);
    for(i=0;i<n/5;i++) used[rand()%n]=1;
    while(1){
        printf("\n1.Show Bit Vector 2.Create New File 3.Show Directory 4.Delete File 5.Exit\n");
        scanf("%d",&ch);
        if(ch==1){for(i=0;i<n;i++)printf("%d ",used[i]);printf("\n");}
        else if(ch==2){
            char name[30];int m;printf("File name and blocks: ");scanf("%s%d",name,&m);
            printf("Enter %d free blocks (for linked/index simulation): ",m);
            if(nf>=MAX){printf("Directory full\n");continue;}
            strcpy(dir[nf].name,name);dir[nf].count=0;
            for(j=0;j<m;j++){scanf("%d",&k);if(k>=0&&k<n&&!used[k]){used[k]=1;dir[nf].blocks[dir[nf].count++]=k;}}
            nf++;
        } else if(ch==3){
            for(i=0;i<nf;i++){printf("%s: ",dir[i].name);for(j=0;j<dir[i].count;j++)printf("%d ",dir[i].blocks[j]);printf("\n");}
        } else if(ch==4){
            char name[30];scanf("%s",name);
            for(i=0;i<nf;i++)if(strcmp(dir[i].name,name)==0){for(j=0;j<dir[i].count;j++)used[dir[i].blocks[j]]=0;dir[i]=dir[--nf];break;}
        } else break;
    }
    return 0;
}
```

Non-preemptive Priority

```
#include <stdio.h>
int main(){
    int n,i,t=0,done=0,idx;scanf("%d",&n);
    int at[n],bt[n],pr[n],ct[n],tat[n],wt[n],used[n];
    for(i=0;i<n;i++){scanf("%d%d%d%d",&at[i],&bt[i],&pr[i]);used[i]=0;}
    printf("Gantt: ");
    while(done<n){
        idx=-1;
        for(i=0;i<n;i++) if(!used[i]&&at[i]<=t&&(idx==-1||pr[i]<pr[idx])) idx=i;
        if(idx==-1){t++;continue;}
        printf("P%d ",idx);t+=bt[idx];ct[idx]=t;tat[idx]=ct[idx]-at[idx];wt[idx]=tat[idx]-bt[idx];used[idx]=1;done++;
    }
    for(i=0;i<n;i++)printf("\nP%d TAT=%d WT=%d",i,tat[i],wt[i]);return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 23

Questions reproduced from page 23 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

fork + bubble/insertion sorting

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
void bubble(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=0;j<n-i-1;j++)if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}}
void insertion(int a[],int n){int i,j,key;for(i=1;i<n;i++){key=a[i];j=i-1;while(j>=0&& a[j]>key){a[j+1]=a[j];j--;}a[j+1]=key;}}
void print(int a[],int n){int i;for(i=0;i<n;i++)printf("%d ",a[i]);printf("\n");}
int main(){
    int n,a[100],b[100],i; scanf("%d",&n);
    for(i=0;i<n;i++){scanf("%d",&a[i]);b[i]=a[i];}
    pid_t p=fork();
    if(p==0){insertion(b,n);printf("Child - Insertion sort: ");print(b,n);}
    else{bubble(a,n);printf("Parent - Bubble sort: ");print(a,n);wait(NULL);}
    return 0;
}
```

Non-preemptive SJF

```
#include <stdio.h>
int main(){
    int n,i,j,t=0,done=0,idx,temp; scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n],used[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);used[i]=0;}
    printf("Gantt: ");
    while(done<n){
        idx=-1;
        for(i=0;i<n;i++) if(!used[i]&&at[i]<=t&&(idx==-1||bt[i]<bt[idx])) idx=i;
        if(idx==-1){t++;continue;}
        printf("P%d ",idx); t+=bt[idx]; ct[idx]=t; tat[idx]=ct[idx]-at[idx]; wt[idx]=tat[idx]-bt[idx]; used[idx]=1;done++;
    }
    float aw=0,aa=0;printf("\nPID AT BT CT TAT WT\n");
    for(i=0;i<n;i++){printf("P%d %d %d %d %d %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 24

Questions reproduced from page 24 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Round Robin

```
#include <stdio.h>
int main(){
    int n,q,i,t=0,done=0;scanf("%d%d",&n,&q);
    int at[n],bt[n],rem[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int progress=0;
        for(i=0;i<n;i++) if(rem[i]>0 && at[i]<=t){
            progress=1;printf("| P%d ",i);
            int x=rem[i]<q?rem[i]:q; t+=x;rem[i]-=x;
            if(rem[i]==0){ct[i]=t;tat[i]=ct[i]-at[i];wt[i]=tat[i]-bt[i];done++;}
        }
        if(!progress)t++;
    }
    printf("\n");float aw=0,aa=0;for(i=0;i<n;i++){printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

fork + execve + binary search

```
/* Main program: sort, then exec child program with sorted values */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void bubble(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=0;j<n-i-1;j++)if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}}
int main(int argc,char *argv[]){
    int n,i; scanf("%d",&n); int a[n]; char *args[105];
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    bubble(a,n);
    char key[20]; scanf("%s",key);
    args[0]="./search"; for(i=0;i<n;i++){args[i+1]=malloc(20);sprintf(args[i+1],"%d",a[i]);}
    args[n+1]=key; args[n+2]=NULL;
    pid_t p=fork();
    if(p==0) execve(args[0],args,NULL);
    else wait(NULL);
    return 0;
}
```

```
/* search.c
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char *argv[]){
    int n=argc-2,key=atoi(argv[argc-1]),l=0,r=n-1,mid;
    while(l<=r){mid=(l+r)/2;int x=atoi(argv[mid+1]);
        if(x==key){printf("Found at position %d\n",mid+1);return 0;}
        if(x<key)l=mid+1;else r=mid-1;
    }
    printf("Not found\n"); return 0;
} */
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 25

Questions reproduced from page 25 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Orphan process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
int main(){
    pid_t pid=fork();
    if(pid<0){perror("fork");return 1;}
    if(pid==0){
        printf("Child: PID=%d, Parent PID=%d\n",getpid(),getppid());
        sleep(5);
        printf("Child after parent termination: PID=%d, New Parent PID=%d\n",getpid(),getppid());
    }else{
        printf("Parent: PID=%d, Child PID=%d\n",getpid(),pid);
        sleep(1);
        printf("Parent terminates.\n");
        exit(0);
    }
    return 0;
}
```

Shell + search command

```
#include <stdio.h>
#include <string.h>
int main(){
    char mode[2],file[100],pat[100],line[500]; FILE *fp;
    printf("$ search <f|a|c> filename pattern\n");
    scanf("%1s %99s %99s",mode,file,pat);
    fp=fopen(file,"r"); if(!fp){perror("fopen");return 1;}
    int count=0,lineNo=0,found=0;
    while(fgets(line,sizeof(line),fp)){
        lineNo++; char *p=line;
        while((p=strstr(p,pat))){
            count++;
            if(mode[0]=='f'){printf("First occurrence: line %d\n",lineNo);found=1;fclose(fp);return 0;}
            if(mode[0]=='a') printf("Occurrence at line %d\n",lineNo);
            p++;
        }
    }
    fclose(fp);
    if(mode[0]=='c') printf("Occurrences = %d\n",count);
    else if(mode[0]=='f' && !found) printf("Pattern not found\n");
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 26

Questions reproduced from page 26 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Preemptive Priority

```
#include <stdio.h>
int main(){
    int n,i,t=0,done=0;scanf("%d",&n);
    int at[n],bt[n],rem[n],pr[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d%d%d",&at[i],&bt[i],&pr[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int idx=-1;
        for(i=0;i<n;i++) if(at[i]<=t&&rem[i]>0&&(idx==-1||pr[i]<pr[idx])) idx=i;
        if(idx==-1){t++;continue;}
        printf("P%d ",idx);rem[idx]--;t++;
        if(rem[idx]==0){ct[idx]=t;tat[idx]=ct[idx]-at[idx];wt[idx]=tat[idx]-bt[idx];done++;}
    }
    printf("\n");for(i=0;i<n;i++)printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);return 0;
}
```

FIFO page replacement

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],i,j,pos=0,fault=0,hit;
    printf("Enter number of references: "); scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++) scanf("%d",&ref[i]);
    printf("Enter number of frames: "); scanf("%d",&f);
    for(i=0;i<f;i++) frame[i]=-1;

    for(i=0;i<n;i++){
        hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]) hit=1;
        if(!hit){ frame[pos]=ref[i]; pos=(pos+1)%f; fault++; }
        printf("%d: ",ref[i]);
        for(j=0;j<f;j++) printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 27

Questions reproduced from page 27 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

SSTF disk scheduling

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int n, req[100], used[100]={0}, head, i, j, idx, dist, best, total=0;
    scanf("%d", &n);
    for(i=0; i<n; i++) scanf("%d", &req[i]);
    scanf("%d", &head);
    printf("Service order: ");
    for(i=0; i<n; i++){
        idx=-1; best=1000000;
        for(j=0; j<n; j++) if(!used[j]){
            dist=abs(head-req[j]);
            if(dist<best){best=dist; idx=j;}
        }
        used[idx]=1; total+=abs(head-req[idx]); head=req[idx];
        printf("%d ", head);
    }
    printf("\nTotal head movement = %d\n", total);
    return 0;
}
```

LRU page replacement

```
#include <stdio.h>
int main(){
    int n, f, ref[100], frame[20], last[20], i, j, t=0, fault=0, hit, idx, min;
    printf("Enter number of references: "); scanf("%d", &n);
    printf("Enter reference string: ");
    for(i=0; i<n; i++) scanf("%d", &ref[i]);
    printf("Enter number of frames: "); scanf("%d", &f);
    for(i=0; i<f; i++) frame[i]=-1, last[i]=-1;

    for(i=0; i<n; i++){
        t++; hit=0; idx=-1;
        for(j=0; j<f; j++) if(frame[j]==ref[i]){
            hit=1; last[j]=t;
        }
        if(!hit){
            fault++;
            for(j=0; j<f; j++) if(frame[j]==-1){ idx=j; break; }
            if(idx==-1){
                min=last[0]; idx=0;
                for(j=1; j<f; j++) if(last[j]<min){min=last[j]; idx=j;}
            }
            frame[idx]=ref[i]; last[idx]=t;
        }
        printf("%d: ", ref[i]);
        for(j=0; j<f; j++) printf("%d ", frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n", fault);
    return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 28

Questions reproduced from page 28 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

SCAN disk scheduling

For this slip use dir=0 for Left.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=i+1;j<n;j++)if(a[i]>a[j]){t=a[i];a[i]=a[j];a[j]=t;}}
int main(){
    int n,a[100],head,dir,i,p=0,total=0;
    scanf("%d",&n); for(i=0;i<n;i++) scanf("%d",&a[i]);
    scanf("%d%d",&head,&dir); /* dir: 0=left, 1=right */
    sort(a,n);
    printf("Service order: ");
    if(dir==0){
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }else{
        for(i=0;i<n;i++) if(a[i]>head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
        for(i=n-1;i>=0;i--) if(a[i]<head){printf("%d ",a[i]);total+=abs(head-a[i]);head=a[i];}
    }
    printf("\nTotal head movement = %d\n",total);
    return 0;
}
```

MFU page replacement

Use the MFU replacement logic below.

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],freq[20],age[20],i,j,hit,idx,minAge,fault=0,t=0;
    scanf("%d",&n); for(i=0;i<n;i++)scanf("%d",&ref[i]); scanf("%d",&f);
    for(i=0;i<f;i++){frame[i]=-1;freq[i]=0;age[i]=0;}
    for(i=0;i<n;i++){
        t++; hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]){hit=1;freq[j]++;age[j]=t;break;}
        if(!hit){
            fault++;idx=-1;
            for(j=0;j<f;j++)if(frame[j]==-1){idx=j;break;}
            if(idx==-1){
                idx=0;
                for(j=1;j<f;j++)
                    if(freq[j]>freq[idx] || (freq[j]==freq[idx]&&age[j]<age[idx])) idx=j;
            }
            frame[idx]=ref[i];freq[idx]=1;age[idx]=t;
        }
        printf("%d: ",ref[i]);for(j=0;j<f;j++)printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault); return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 29

Questions reproduced from page 29 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

fork + execve + binary search

```
/* Main program: sort, then exec child program with sorted values */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void bubble(int a[],int n){int i,j,t;for(i=0;i<n-1;i++)for(j=0;j<n-i-1;j++)if(a[j]>a[j+1]){t=a[j];a[j]=a[j+1];a[j+1]=t;}}
int main(int argc,char *argv[]){
    int n,i; scanf("%d",&n); int a[n]; char *args[105];
    for(i=0;i<n;i++) scanf("%d",&a[i]);
    bubble(a,n);
    char key[20]; scanf("%s",key);
    args[0]="./search"; for(i=0;i<n;i++){args[i+1]=malloc(20);sprintf(args[i+1],"%d",a[i]);}
    args[n+1]=key; args[n+2]=NULL;
    pid_t p=fork();
    if(p==0) execve(args[0],args,NULL);
    else wait(NULL);
    return 0;
}

/* search.c
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char *argv[]){
    int n=argc-2,key=atoi(argv[argc-1]),l=0,r=n-1,mid;
    while(l<=r){mid=(l+r)/2;int x=atoi(argv[mid+1]);
        if(x==key){printf("Found at position %d\n",mid+1);return 0;}
        if(x<key)l=mid+1;else r=mid-1;
    }
    printf("Not found\n"); return 0;
} */
```

FCFS CPU scheduling

```
#include <stdio.h>
int main(){
    int n,i,j,t=0; printf("Enter n: ");scanf("%d",&n);
    int at[n],bt[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){printf("AT and CPU burst of P%d: ",i);scanf("%d%d",&at[i],&bt[i]);}
    for(i=0;i<n;i++){ if(t<at[i]) t=at[i]; t+=bt[i]; ct[i]=t; tat[i]=ct[i]-at[i]; wt[i]=tat[i]-bt[i];}
    printf("Gantt: |");for(i=0;i<n;i++)printf(" P%d |",i);
    printf("\nPID AT BT CT TAT WT\n");
    float aw=0,atot=0;
    for(i=0;i<n;i++){printf("P%d %d %d %d %d %d\n",i,at[i],bt[i],ct[i],tat[i],wt[i]);aw+=wt[i];atot+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,atot/n); return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.

SLIP 30

Questions reproduced from page 30 of the uploaded slip booklet. ■filecite■turn0file0■L1-L8■

Round Robin CPU scheduling

```
#include <stdio.h>
int main(){
    int n,q,i,t=0,done=0;scanf("%d%d",&n,&q);
    int at[n],bt[n],rem[n],ct[n],tat[n],wt[n];
    for(i=0;i<n;i++){scanf("%d%d",&at[i],&bt[i]);rem[i]=bt[i];}
    printf("Gantt: ");
    while(done<n){
        int progress=0;
        for(i=0;i<n;i++) if(rem[i]>0 && at[i]<=t){
            progress=1;printf("| P%d ",i);
            int x=rem[i]<q?rem[i]:q; t+=x;rem[i]-=x;
            if(rem[i]==0){ct[i]=t;tat[i]=ct[i]-at[i];wt[i]=tat[i]-bt[i];done++;}
        }
        if(!progress)t++;
    }
    printf("\n");float aw=0,aa=0;for(i=0;i<n;i++){printf("P%d TAT=%d WT=%d\n",i,tat[i],wt[i]);aw+=wt[i];aa+=tat[i];}
    printf("Average WT=%.2f Average TAT=%.2f\n",aw/n,aa/n);return 0;
}
```

MFU page replacement

Use the MFU replacement logic below.

```
#include <stdio.h>
int main(){
    int n,f,ref[100],frame[20],freq[20],age[20],i,j,hit,idx,minAge,fault=0,t=0;
    scanf("%d",&n); for(i=0;i<n;i++)scanf("%d",&ref[i]); scanf("%d",&f);
    for(i=0;i<f;i++){frame[i]=-1;freq[i]=0;age[i]=0;}
    for(i=0;i<n;i++){
        t++; hit=0;
        for(j=0;j<f;j++) if(frame[j]==ref[i]){hit=1;freq[j]++;age[j]=t;break;}
        if(!hit){
            fault++;idx=-1;
            for(j=0;j<f;j++)if(frame[j]==-1){idx=j;break;}
            if(idx==-1){
                idx=0;
                for(j=1;j<f;j++)
                    if(freq[j]>freq[idx] || (freq[j]==freq[idx]&&age[j]<age[idx])) idx=j;
            }
            frame[idx]=ref[i];freq[idx]=1;age[idx]=t;
        }
        printf("%d: ",ref[i]);for(j=0;j<f;j++)printf("%d ",frame[j]);
        printf(hit?" Hit\n":" Fault\n");
    }
    printf("Total page faults = %d\n",fault); return 0;
}
```

Exam notes

Compile on Linux with GCC, e.g. gcc program.c -o program. System-call programs require Linux/WSL. Replace random CPU-burst generation with rand() if your examiner specifically asks for random next bursts.